

SIMATSENGINEERING(SAVEETHASCHOOLOFENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 - ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO2 - Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 - Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	THEJAS B	Reg. No.:	192565092
Section/Batch:	2025	Date:	20/07/2026

Code of Conduct

I THEJAS B _____ (Name/RegNo)
 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I
 understand that any violation of academic integrity rules will result in disciplinary action.
 Signature of the Student: THEJAS B

Instructions

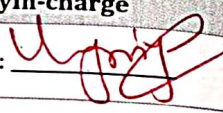
- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answers using an Analytical problem-solving, in evaluation the answers should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section/Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:			100 Marks

Annexure A - Analytical Problem Solving

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Name : Thejas B

Reg no: 192665092

Cowise: CSA0305

Cowsecode: Data structure.

1. Reordering a singly linked list $L: L_1 \rightarrow L_2 \rightarrow L_3 \dots \rightarrow L_{n-1} \rightarrow L_n$,
reorder and print it to: $L_1 \rightarrow L_n \rightarrow L_2 \rightarrow L_{n-1}$.

Aim:

To reorder a singly linked list form:

$L_1 \rightarrow L_2 \rightarrow L_3 \rightarrow \dots \rightarrow L_n$

to

$L_1 \rightarrow L_n \rightarrow L_2 \rightarrow L_{n-1} \rightarrow \dots$

Algorithm:

1. Start with the head node.
2. Find the middle of the linked list using slow and fast pointers.
3. Reverse the second half of the list.
4. Merge the first half and reversed second half alternately.
5. Print the reordered list.

Pseudocode:

Findmiddle(head)

Reverse(secondhalf)

Merge(firsthalf, secondhalf)

Display()

Example

Before Reordering

$10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 50$

After Reordering

$10 \rightarrow 50 \rightarrow 20 \rightarrow 40 \rightarrow 30$.

Time Complexity

- Finding middle : $O(n)$
- Reversing list : $O(n)$
- Merging : $O(n)$
- Overall : $O(n)$

Advantages:

- Efficient with linear time complexity
- No extra memory required.
- Suitable for large linked lists.

Application :

- Data Rearrangement.
- Playlist Management.
- memory Organization.

Result:

The singly linked list is successfully reordered.

2) Doubly Linked List Insertion and Deletion.

Aim:

To insert 150, 250, 350, 450, 550, into a doubly linked list and delete 150 and 250.

Algorithm.

Insertion.

1. Create a new node.
2. If the list is empty, make it the head.
3. Otherwise, insert the node at the end.
4. Repeat for all elements.

Deletion:

1. Search for the node.
2. update previous and next links.
3. Delete the node.
4. Repeat for the next node.

Pseudocode.

Insert (data)

Delete (data)

Display ()

Advantages:

- * Supports forward and backward traversal.
- * Easier insertion and deletion than singly linked lists.
- * Efficient for navigation.

Applications

- * Browser history.
- * Music playlist.
- * Undo / Redo Operations.
- * Image viewers.

Result :

The elements 150 and 250 are deleted successfully.

Work